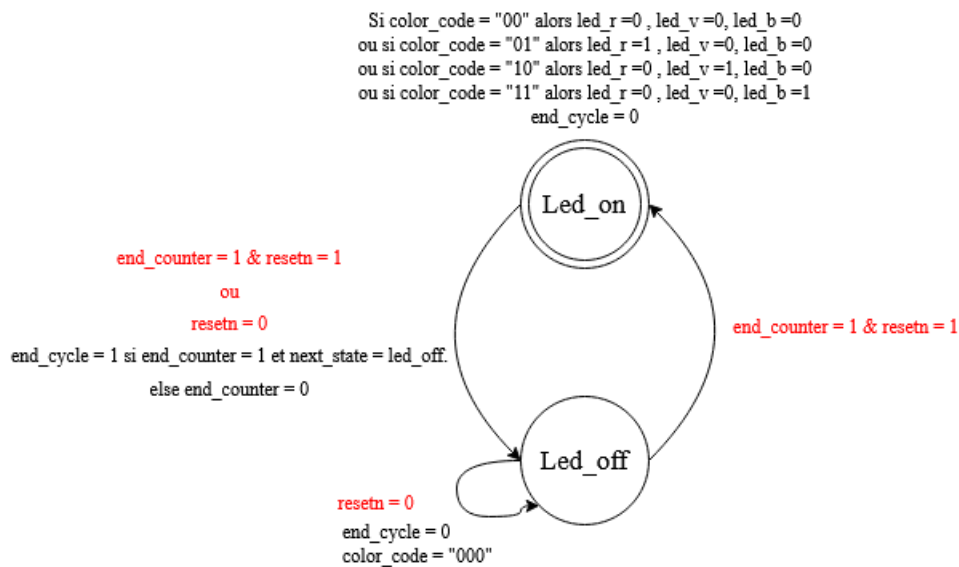


17/07/2026 TP4 : Led_Driver et mémoire FIFO.

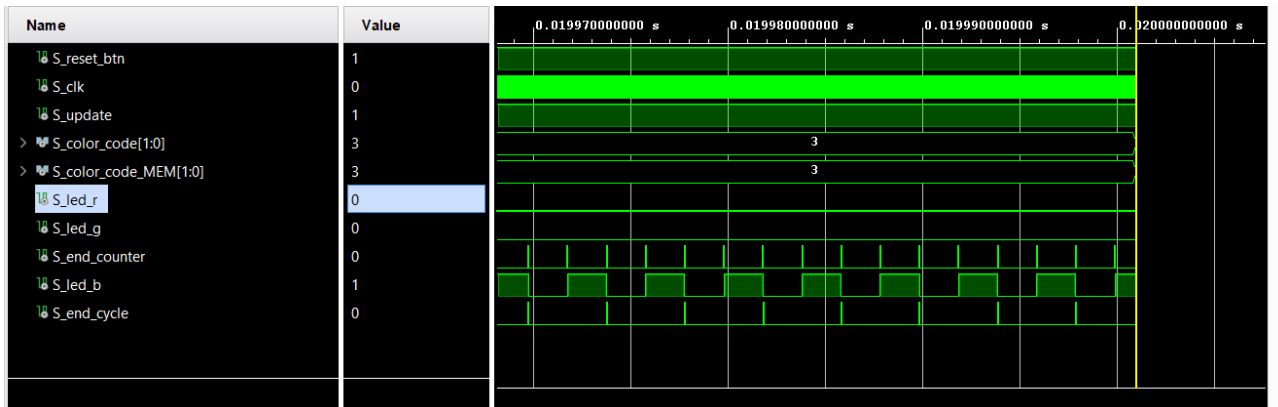
Livrables
Schéma RTL
Résultat de simulation avec chronogramme commentés
Vos résultats de synthèse (analyse de vos ressources)
Vos résultats de STA (analyse du rapport de timing)
Une démonstration de votre design

L'objectif de cette partie est de réaliser un design permettant de faire clignoter une LED RGB avec une séquence de couleurs entrées à l'aide des boutons. Dans cette partie, vous utiliserez le module LED_driver de la partie 1 et vous ajouterez l'utilisation d'un composant mémoire : la FIFO

1. Sur l'architecture RTL, modifiez le module LED_driver en ajoutant une sortie end_cycle. Cette sortie vaudra 1 à la fin d'un cycle allumé/éteint de la LED RGB.



End_cycle doit être un pulse lorsque end_counter vaut 1 et qu'un nouveau cycle d'allumage est commencé



S_end_cycle vaut 1 à chaque passage de l'état led_on à led_off, une fois que la led à clignotée, au début de chaque nouveau cycle.

2. Modifiez la logique en entrée du module pour ajouter une FIFO. Cette FIFO doit prendre en entrée le **code couleur « vert » ou « bleu » suivant l'état du bouton_1** et est connectée en sortie à l'entrée **color_code** du module **LED_driver**. La donnée est écrite dans la FIFO lorsqu'il y a un **front montant du bouton_0**. La donnée de la FIFO est lue **lorsque le signal end_cycle du module LED_driver vaut 1**.

a) Description du fonctionnement de la fifo :

Block RAM FIFOs Versus Built-in FIFOs :

La solution built-in Fifo avec une horloge commune sur la partie lecture et écriture à été choisie.

Block ram are 36 Kb of RAMs placed in columns within the clock regions and across the device in UltraScale architectures, block ram are used for large data arrays.

Distributed RAM provides trade-off between using storage elements for very small arrays and block RAM. This kind of memory should be used for data that are 64 bits or less.

Built in fifo, every component are instantiated within the programmable logic array.

Table 1-1: Memory Configuration Benefits

	Independent Clocks	Common Clock	Small Buffering	Medium-Large Buffering	High Performance	Minimal Resources
Built-in FIFO	✓	✓		✓	✓	✓
Block RAM	✓	✓		✓	✓	✓
Shift Register		✓	✓		✓	
Distributed RAM	✓	✓	✓		✓	

cf FIFO Generator v13.2 Product Guide (PG057) P22

Name	Direction	Description
Rst	Input	Reset: An asynchronous reset that initializes all internal pointers and output registers.
clk	Input	Clock: All signals on the write and read domains are synchronous to this clock.
din[n:0]	Input	Data Input: The input data bus used when writing the FIFO.
wr_en	Input	Write Enable: If the FIFO is not full, asserting this signal causes data (on din) to be written to the FIFO.
full	Output	Full Flag: When asserted, this signal indicates that the FIFO is full. Write requests are ignored when the FIFO is full, initiating a write when the FIFO is full is not destructive to the contents of the FIFO
dout[m:0]	Output	Data Output: The output data bus driven when reading the FIFO
rd_en	Input	Read Enable: If the FIFO is not empty, asserting this signal causes data to be read from the FIFO (output on dout).
empty	Output	Empty Flag: When asserted, this signal indicates that the FIFO is empty. Read requests are ignored when the FIFO is empty, initiating a read while empty is not destructive to the FIFO.

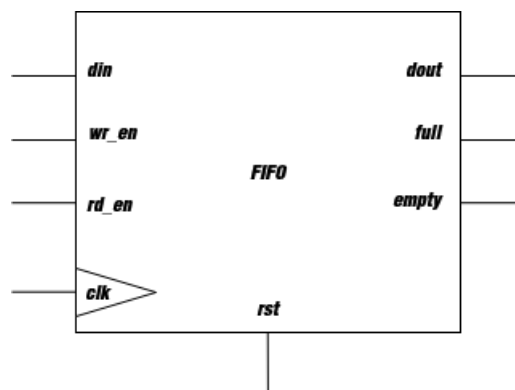


Schéma du block FIFO.

- Choisir une horloge commune pour la lecture et l'écriture semble être une solution plus simple que de gérer deux horloges, de plus tout notre système est synchrone.
- J'ai choisi « built-in FIFO » car il s'agit d'une solution peu consommatrice de ressource, utiliser la block ram semble être également une bonne solution.
- J'ai choisi de stocker la variable colorcode qui est un vecteur de deux bits. Afin que le module led_driver puisse directement lire cette variable. Write width and read width ont été choisis égaux à 2.
- la taille du buffer est minimale, choisir une séquence de 512 clignotements semble suffisant pour répondre au besoin.
- Reset Asynchrone comme les autres éléments de notre architecture.
- Les entrées sorties facultatives n'ont pas été conservées par manque de besoin.

Estimated FIFO Usage : 1 of type: 512x36

Number of FIFO chained in depth : 1

Clocking Scheme	Common Clock
Memory Type	Built-in FIFO
Model Generated	Behavioral Model
Write Width	2
Write Depth	512
Read Width	2
Read Depth	512
Almost Full/Empty Flags	Not Selected/Not Selected
Programmable Full/Empty Flags	Not Selected/Not Selected
Data Count Outputs	Not Selected
Handshaking	Not Selected
Read Mode / Reset	Standard FIFO / Asynchronous
Read Latency (From Rising Edge of Read Clock)	1

Si l'on s'en tient à l'énoncé, la dernière diode choisie va continuer à clignoter ce qui n'est pas conforme à l'objectif de faire clignoter une séquence programmée par l'utilisateur.

On rajoute la condition suivante : lorsque la fifo est vide (flag empty levé) color_code est égal à 0 ; on éteint les diodes.

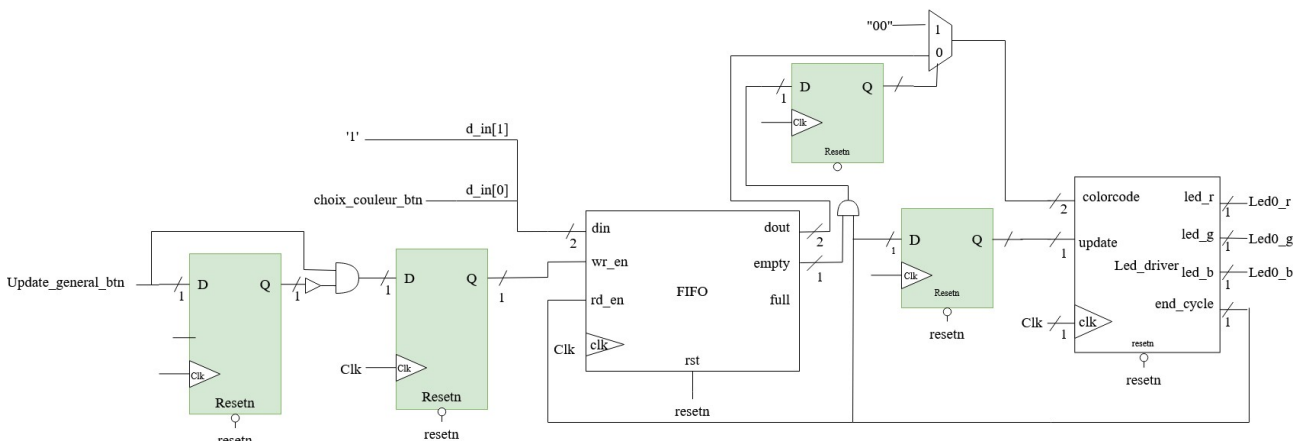


Schéma RTL du bloc mémoire Fifo et du bloc de pilotage des leds.

A chaque fin de cycle, le signal *end_cycle* est envoyé à la fifo sur *rd_en* pour rendre disponible le prochain état de *dout* disponible dans la mémoire, *dout* est actualisé lorsque *rd_en* repasse à l'état bas, il faut donc retarder d'un coup d'horloge le signal de mis à jour de *led_driver*, ce qui est fait à l'aide d'une bascule.

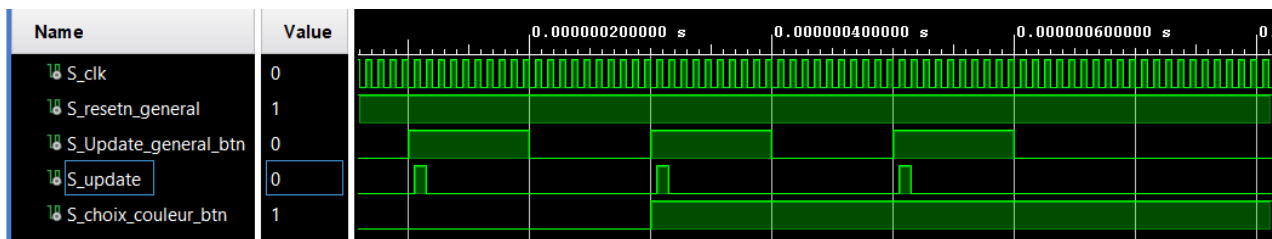
Si le flag *empty* de la fifo est levé et que l'on termine un cycle, il faut éteindre toutes les leds en chargeant « 00 » dans *color_code*, de nouveau, une bascule permet de synchroniser le signal d'update avec la bonne information.

3. Modifiez vos codes de la partie 1 pour y ajouter les nouveaux éléments de votre architecture.

4. Mettez à jour le testbench et réalisez une simulation pour vérifier votre design.

Elément testé	Critère de validation	Ok
Bouton Update	1 unique pulse généré sur pression du bouton Update.	OK
Mémorisation de la FIFO	Le code sur Din est mémorisé sur chaque front montant de wr_en.	OK
Restitution des données	Sur chaque front montant de rd_en, Dout s'actualise avec la séquence mémorisée.	OK
Extinction des leds	Si empty et end cycle vaut 1, on actualise la mémoire interne de led driver avec « 00 »	OK
Reset	Si resetn vaut 0, La mémoire de la fifo est réinitialisé et led_drive perd sa valeur de S_color_mem.	OK

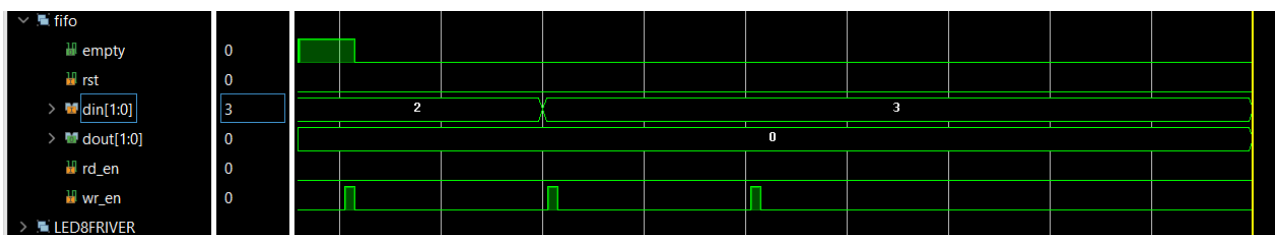
A/ Bouton Update



A chaque fois que le bouton update est pressé, un unique pulse est généré, le comportement est conforme aux attentes.

S_choix_couleur_btn est le signal associé au bouton et au lsb de din, la séquence ci-dessus nous renseigne sur la séquence que l'on enregistre dans la mémoire de la fifo : « 10;11;11 ».

B/ Mémorisation de la FIFO



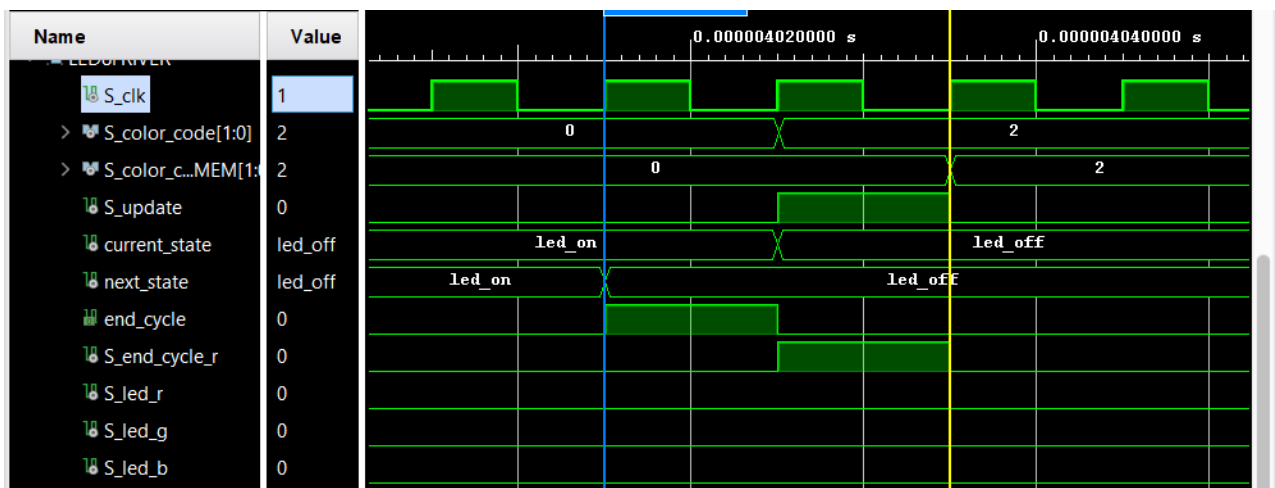
Wr_en est l'image de S_update, din change de valeur lorsque le bouton de sélection de la couleur est enfoncé. le flag empty se baisse lorsque wr_en vaut 1 ; On rentre dans la mémoire la bonne séquence. Dout est initialisé à 0, aucun signal demandant de rendre disponible le contenu de la fifo. Il reste à 0.

C/ Restitution des données

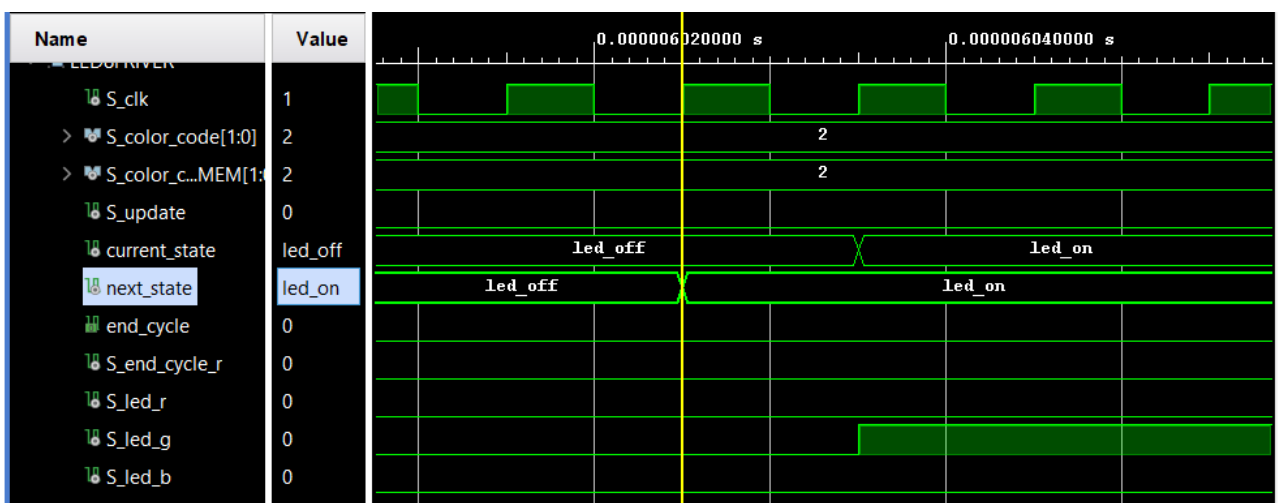


rd_en passe à l'état haut à chaque fin de cycle, les marqueurs bleus indiquent cette mise à l'état haut. On constate que la séquence disponible sur dout vaut 2,3,3 conformément au résultat attendu. Le flag empty, passe à l'état haut après le troisième front montant sur rd_en indiquant que la mémoire est vide.

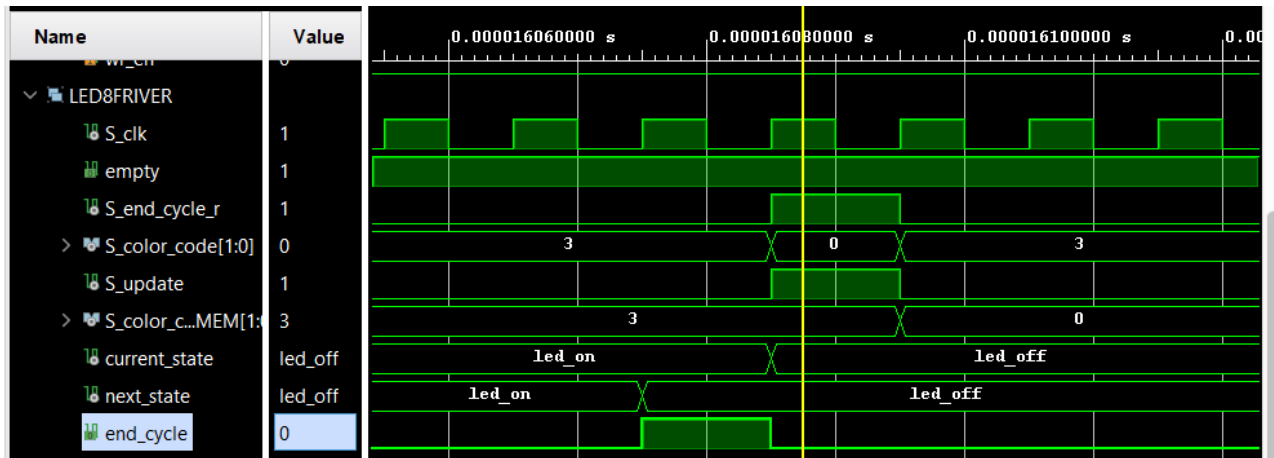
D/ Allumage des leds.



Sur end_cycle, la sortie de la fifo est actualisée, S_color_code, prend comme valeur Dout, le signal d'update permet de charger S_color_code dans la mémoire de led_driver S_color_code_mem. Les diodes s'allument lorsque l'état devient led_on.

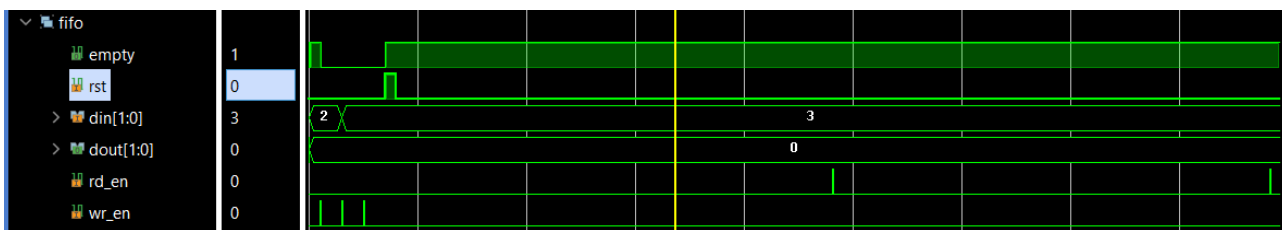


E/ Extinction des leds lorsque la mémoire est vide.

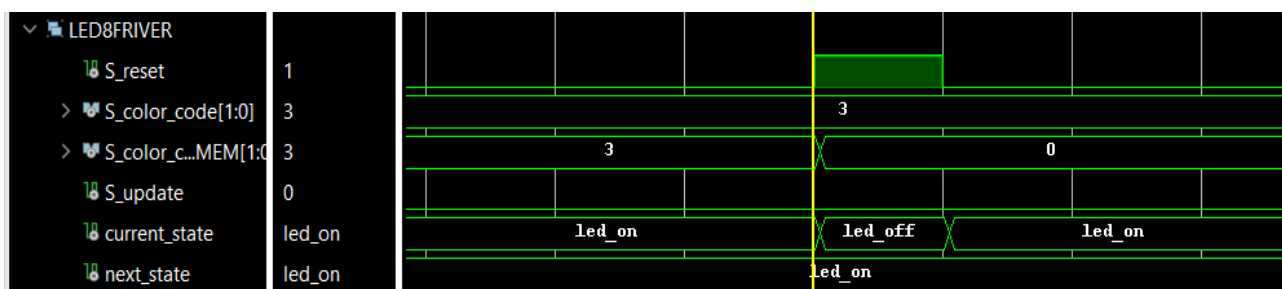


Si empty vaut 1 et S_end_cycle (retardé d'un coup d'horloge) vaut 1, S_color_code est mis à '00' afin de charger dans la mémoire de led_driver (S_color_code_mem) la commande permettant d'extinction des leds.

F/ Reset

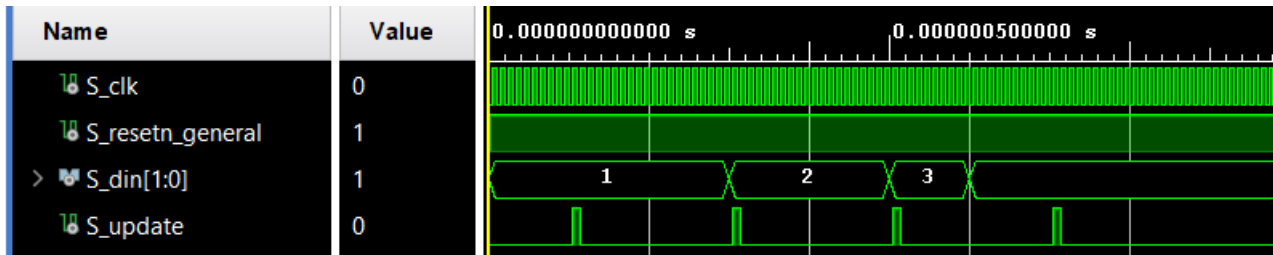


Sur reset actif, la mémoire de la fifo se vide car le flag empty se met à l'état haut, la séquence qu'on l'on a rentré sur Din, ne déroule pas lorsqu'on essaie de la lire ensuite.

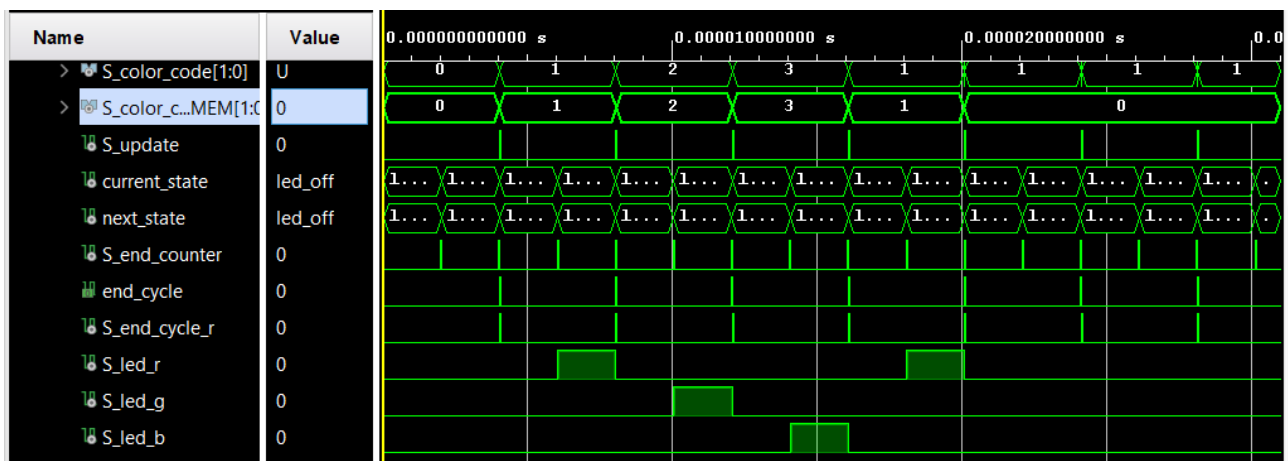


Sur reset actif, current state devient led_off et S_color_mem est à 0.

G/ Test avec un vecteur d'entrée à la place du bouton de sélection de la led pour pouvoir tester la led rouge.

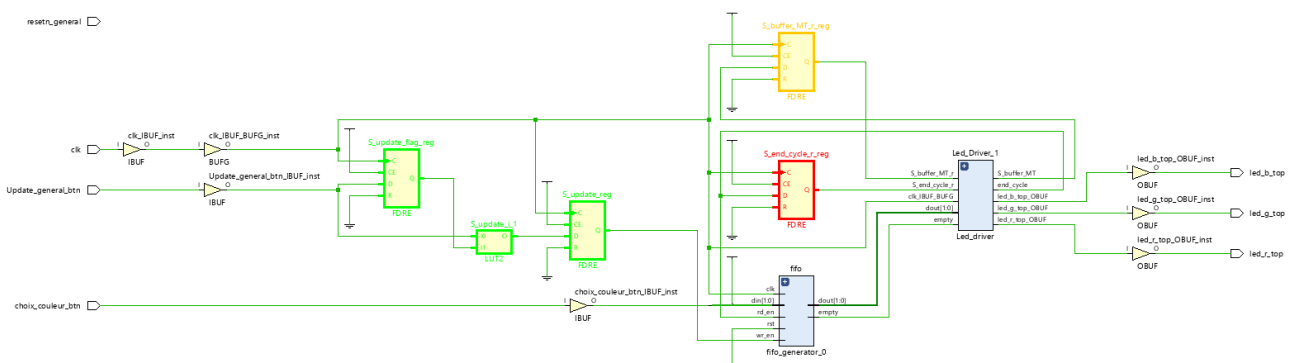


S_din est le vecteur d'entrée, on charge la séquence suivante : 01,10,11,01 soit 1,2,3,1.
On attend en sortie la séquence suivante : rouge, vert, bleu, rouge.



La bonne séquence est retransmise en sortie, puis les diodes s'éteignent car la mémoire est vide.

5. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.



- Les deux bascules et la lut surlignées en vert génèrent le pulse du bouton Update.
- On retrouve les deux bascules de 'temporisation' permettant de retarder la mis à jour de led driver. (Bascule rouge et orange).
- Le multiplexeur permettant d'initialiser color_code à « 00 » à été placé dans led_driver (composant surligné en rose) Cf schématique du projet en pj.


```

Report Cell Usage:
+-----+-----+-----+
|      |Cell      |Count |
+-----+-----+-----+
|1      |fifo_generator |    1|
|2      |BUFG           |    1|
|3      |CARRY4         |    7|
|4      |LUT2           |   36|
|5      |LUT3           |    5|
|6      |LUT4           |    3|
|7      |LUT6           |    3|
|8      |FDRE           |   37|
|9      |LDC            |    2|
|10     |LDCP           |    1|
|11     |IBUF           |    3|
|12     |OBUF           |    3|
+-----+-----+-----+

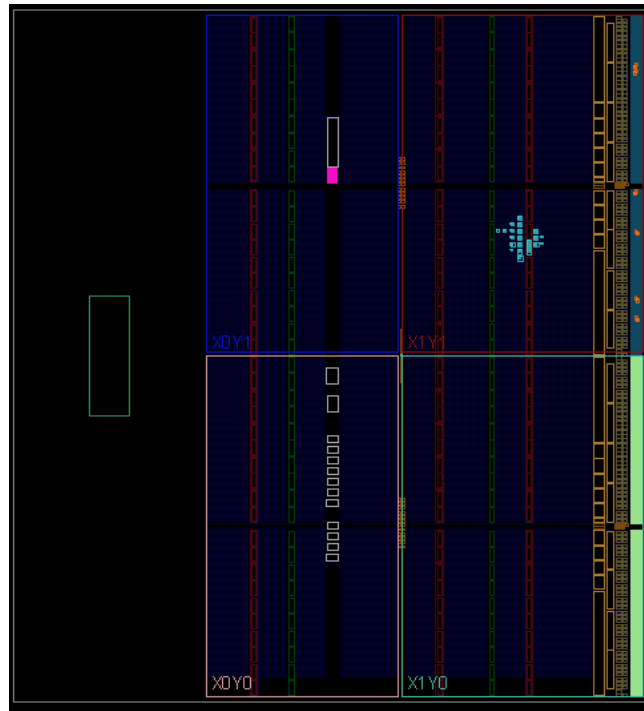
```

Nom entité	Sous entité	FDRE/LDC
Compteur de temporisation	Mémorisation V_data	28 FDRE
Driver de led	Machine à état	2 FDRE
Driver de led	Sortie des leds	3 LDC
Driver de led	Fin de cycle	1 FDRE
Driver de led	Mémorisation color_code	2 FDRE
Pilotage	Temporisation	2 FDRE
Bouton entrée	Détecteur de front montant	2 FDRE

3 Ibufs pour les deux boutons d'entrées et l'horloge.

3 obufs pour les trois leds.

6. Effectuez le placement routage et étudiez les rapports.



Occupation des slices sans ILA.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3,131 ns	Worst Hold Slack (WHS): 0,160 ns	Worst Pulse Width Slack (WPWS): 3,020 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 42	Total Number of Endpoints: 42	Total Number of Endpoints: 51
All user specified timing constraints are met.		

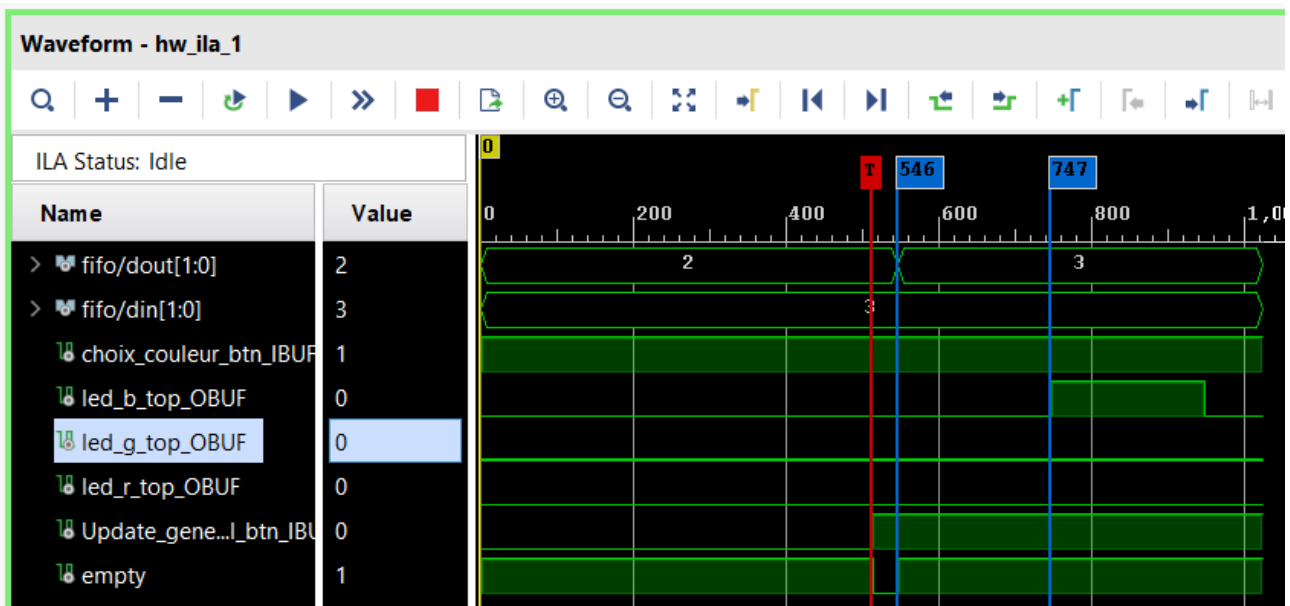
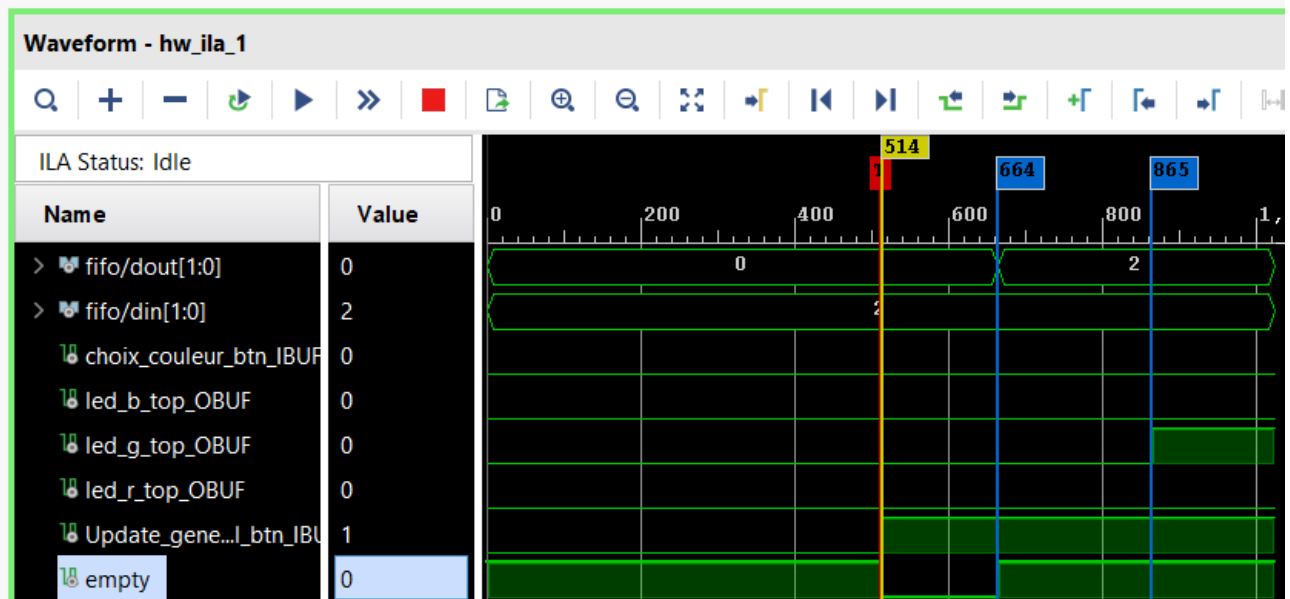
Tout les slacks sont positifs.

7. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

Oui, on obtiens bien le comportement attendu sur carte. Les deux boutons permettent de sélectionner une séquence d'allumage qui est retranscrite par intervalle de deux secondes.
Cf : Vidéo de démonstration.

A/ Utilisation de l'ILA pour valider l'allumage des diodes.

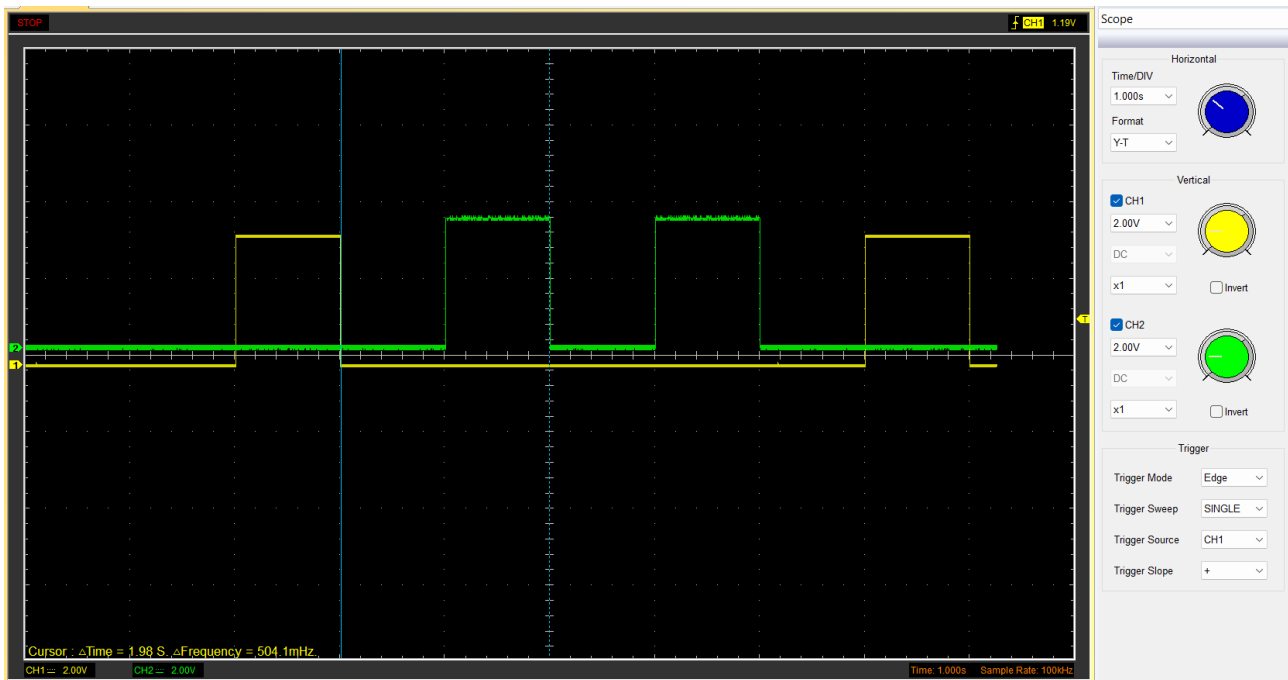
Validation fonctionnement de la prise en compte du chargement de la diode avec un délais de temporisation de 200 coups d'horloge.



B/ Utilisation de l'oscilloscope pour échantillonner une séquence d'allumage de leds.

On vise une vitesse de clignotement à 2seconde par cycle de clignotement.

Nb_coup_tempo = 1*120E6



Séquence vert, bleu, bleu, vert enregistré à l'oscilloscope avec un délais de temporisation de 1s.

Ressources consultées :

AMD IP Fifo documentation : <https://docs.amd.com/v/u/en-US/pg057-fifo-generator>